Cost method, step by step

Beta · 2026-10-14-students-1, 14 October 2026

Beta: this resource may still change before its session. The site is updated in place; the pack version above identifies what you read.

> Beta. Pack 2026-10-14-students-1. Set your machine up with Before TD 1.

The required lab calls use free routes. Record zero real spend only after checking the plan and route actually used, with dated evidence. The cost is computed all the same, because the decision made in TD 3 is about what the same configuration would cost a company. This document gives the procedure, from reading the tokens to the annual bill.

Reading note: all amounts exclude tax. Prices per million tokens are in dollars, subscription costs in euros.

The three quantities, never added together

Quantity	What it measures	Where it comes from
Real spend	What the sub-group pays during the lab session	Native cost if provided; otherwise documented plan/ route evidence, separate from the raw trace field
API-equivalent cost	What the same consumption would cost on a route billed per token	The tokens of the trace multiplied by a dated price
Cost charged to a subscription	The share of the fee consumed by that call	The price of the fee divided by the consumption of the period

These three columns stay separate in every table you produce. A total covers one of them only.

Step 1 - Read the tokens in the trace

Open `traces-td1.csv` and read, for the chosen call: `model_requested`, `tokens_prompt`, `tokens_completion`, `tokens_reasoning`, `cost_native`, `latency_ms`, `date_utc`.

Count output tokens once. When the provider includes reasoning in `tokens_completion`, `tokens_reasoning` is information only: do not add it. When the provider counts it separately, add it to the output tokens. The provider documentation settles this point.

An empty counter stays empty. Write N/A, never zero: zero is a measurement, a missing measurement is not.

Step 2 - Choose the reference price

Choose a documented billed route of **the exact same model**, then record input/output prices, currency, date and source. The following Groq prices were checked on the official model catalogue on **5 October 2026**, in USD excluding tax per million tokens:

Model in the kit	Reference billed route	Input	Output
`openai/gpt-oss-120b`	Groq Developer	0.15	0.60
`openai/gpt-oss-20b`	Groq Developer	0.075	0.30
`qwen/qwen3.8-27b`	Groq Developer	0.80	4.00
`cohere/north-mini-code:free`	exact matching paid route not documented in this kit	N/A	N/A
`dots-studio/dots-3-note-preview:free`	exact matching paid route not documented in this kit	N/A	N/A

The dated portfolio remains a reference snapshot. The operational selection above is dated 14 October 2026. For Cohere North Mini Code and Dots3-Note Preview, the OpenRouter catalogue, checked on that date, documents a free route without a matching paid route. Keep the billed equivalent N/A until a source documents the exact same model and dated prices. A shared family name does not establish model identity.

Step 3 - Compute one call

```
API-equivalent cost in USD excl. tax = (input_tokens x input_price + output_tokens
x output_price) / 1,000,000
```

Fictional Groq example, unrelated to a lab question: `openai/gpt-oss-120b`, 2,000 input tokens and 300 output tokens, `cost_native` absent in the response. The account's Free plan has been checked and recorded for this example; the billing documentation, checked on 5 October 2026, distinguishes Free from usage-billed Developer.

Quantity	Value and evidence
Native cost in the raw trace	Empty: the provider supplied no measurement
Real spend	0 USD excl. tax, established from the verified Free plan, not from the missing native field
API equivalent	(2,000 x 0.15 + 300 x 0.60) / 1,000,000 = **0.00048 USD excl. tax**, using Groq Developer prices of 5 October 2026

For your own call, keep the actual tokens and dated plan evidence. If the plan cannot be verified, real spend is N/A. A derived zero belongs in the cost table with its source; the missing raw field remains empty.

This computation assumes plain text, with no context cache and no separately billed tool. If your trace carries one of these categories, write it as an assumption under the table.

Step 4 - Compute one configuration

Add up **every call made**, retries included. A quorum counts each model consulted. An escalation counts the calls already issued before it. The last attempt settles the verdict; all of them consumed capacity and contribute to the equivalent when it can be computed.

Configuration	Cases handled	Answers / abstentions / errors	Calls and retries	Input / output tokens	API-equivalent cost	Real spend
Single model						
2-of-3 quorum						
3-of-5 quorum						
Your routing rule						

For a complete equivalent, first verify that every included attempt has the required tokens and exact-model prices. If a configuration is only partly quantifiable, mark its total N/A, state coverage (known calls/total calls), name the missing models or counters, and label any known subtotal explicitly. Do not project that subtotal as the full configuration cost.

Then divide a complete equivalent by the number of requests handled to obtain a cost per request. State the abstentions and the errors next to it: a cheaper configuration that abstains often does not deliver the same service.

A replayed row keeps the counters of the original run. It is not a new consumption and does not enter the total for the day.

Step 5 - Move to the scale of a company

State the annual volume, then multiply by the cost per request:

```
20 people x 10 requests per day x 20 working days x 12 months = 48,000 requests per year
```

Average cost per request	Annual bill
0.03 USD excl. tax	1,440 USD excl. tax
3 USD excl. tax	144,000 USD excl. tax
30 USD excl. tax	1,440,000 USD excl. tax

The gap between these three rows does not come from the volume, which is identical, but from the model and the configuration chosen for each request. That is the subject of the routing rule of TD 2.

A complex request handled by a latest-generation model chains several calls: one hundred calls of 20,000 input tokens and 2,000 output tokens, at prices of 10 and 50 dollars excluding tax per million, cost 30 dollars excluding tax. That amount is a computation under written assumptions, not a measured average cost.

What this method does not give

It does not give a provider invoice: real billing adds the context cache, the tools, volume discounts and long-context surcharges. It does not give an error rate per model: that requires repeated questions and an independent sample, outside these three sessions. Twenty questions also establish no permanent model ranking or error bound guaranteed by a quorum. It does not give the cost of the human work around the system either, which TD 3 mentions without quantifying.

Appendix - Computation errors to correct during the session

- A cost shown as zero when the price was never entered: the column stays empty, it does not equal zero.
- Reasoning tokens added to output tokens that already contained them: the total doubles for no reason.
- An API-equivalent cost presented as a spend: the two columns do not mix.
- A price with no date: the catalogue has changed since, and the computation can no longer be checked.